

Team Details

Team Name

Eclipse

Team Member 1

Name - Parth Singla

*College - Indian Institute of Technology,
Patna (IITP)*

Roll No. - 2401CS18

Email Id - parth_2401cs18@iitp.ac.in

*Degree & Dept. - Computer Science and
Engineering (CSE)*

Year - 2nd year moving to 3rd year

Team Member 2

Name - Aditya Raj

*College - Indian Insititute of Technology,
Patna (IITP)*

Roll No. - 2401MC56

Email Id - aditya_2401mc56@iitp.ac.in

*Degree & Dept. - Mathematics and
Computing (MNC)*

Year - 2nd year moving to 3rd year

Problem Statement

Selected Problem

Problem Statement-3

Context-Aware, Adaptive Memory Solution for Mobile Agentic Systems

Problem Understanding

Modern smartphones run increasingly memory-hungry applications — particularly on-device GenAI models. Traditional memory managers like Android's Imkd use simple LRU (Least Recently Used) eviction, which is reactive and context-blind: it doesn't know which app the user will open next, so it evicts apps that are frequently needed, causing thrashing (evict → reload → evict cycles), slow cold starts (50–100ms page faults), and unnecessary battery drain from repeated storage I/O. The core problem is: how do you keep the right apps in RAM before the user asks for them, especially when GenAI workloads leave almost no memory headroom?

Why We Selected It:

1. **ML meets Systems:** The problem sits at a fascinating intersection — you need ML (sequence prediction) to anticipate user behavior, but the solution must run under extreme constraints: <1.5ms inference latency, <100KB model size, millijoule-level energy budgets. This forced us to think about edge-native design (T-GRU instead of Transformers, ONNX export for NPU, ZRAM compression tiers) rather than just throwing a bigger model at the problem.²
2. **Measurable impact:** Unlike many ML projects, this one has hard KPIs — cache hit rate, load time reduction, thrashing elimination, battery savings — that can be directly benchmarked against the existing LRU baseline. Our system achieved 6/6 KPIs, including -41% load time and -74% thrashing under 2GB memory pressure, proving the approach works precisely where it's needed most.

Proposed Solution

Our solution replaces the static LRU eviction policy with a prediction-driven, multi-tier memory manager that anticipates user behavior and proactively optimizes memory allocation.

1. Predict Next App

- T-GRU + Markov Ensemble — predicts which app opens next with 81% Top-3 accuracy
- Time-decay gating flushes stale context after idle periods
- Exported as 82KB ONNX model for NPU inference in <1.5ms

2. Smart Memory Management

- ARC Eviction: Self-tuning score = recency + frequency + ML_prediction replaces static LRU
- Tiered Preloading + Page Clustering: Only preload the startup pages (25% of app binary), not the full APK — 3 tiers based on prediction confidence
- ZRAM Compressed Tier: Evicted apps → LZ4 compressed RAM (0.5ms restore vs 50ms from storage, 80% hit rate)

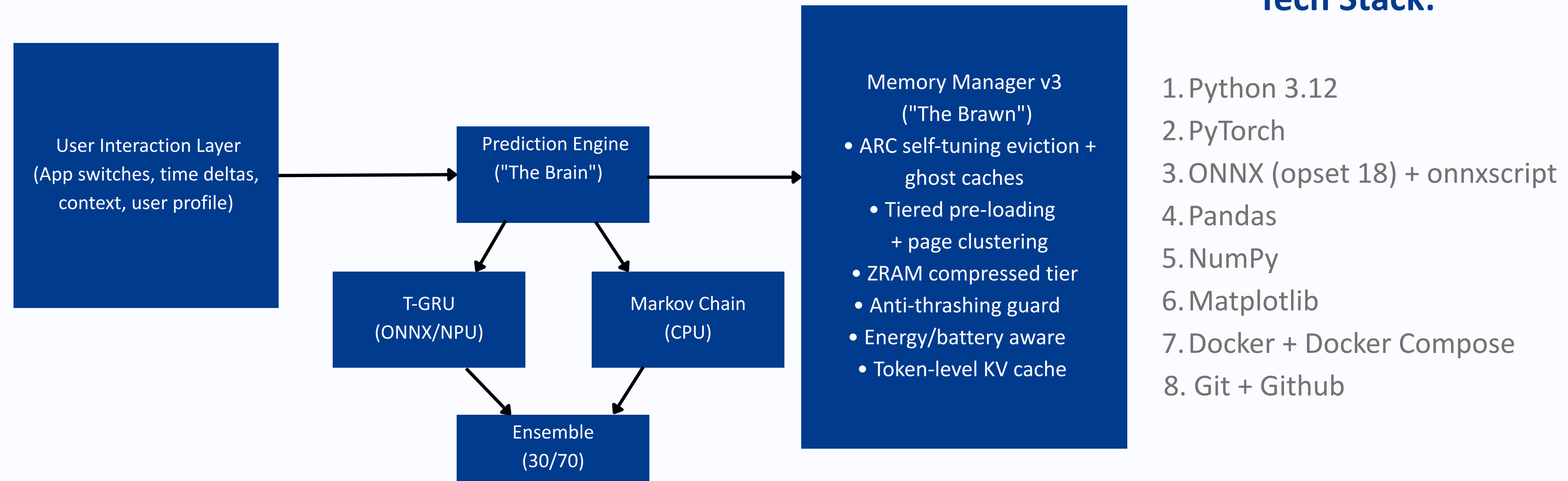
3. Sim-to-Real Deployment Path

- Android: Hook into Imkd (Low Memory Killer Daemon) to replace its LRU policy with our scoring function. ONNX model runs on Samsung's NPU via ONE/NNAPI.
- Linux: Use madvise() syscalls for tiered preloading and cgroups v2 for per-app memory budgeting.
- Energy-Aware: All operations are profiled in millijoules using ARM Cortex-A78 power estimates, achieving 6.5% battery savings by reducing storage I/O.

4. Dockerization

The system is fully Dockerized for one-command reproducibility: `docker build -t edge-memory-sim .` && `docker run edge-memory-sim`.

Architecture Diagram + Tech Stack



Technical Details (Prediction Engine)

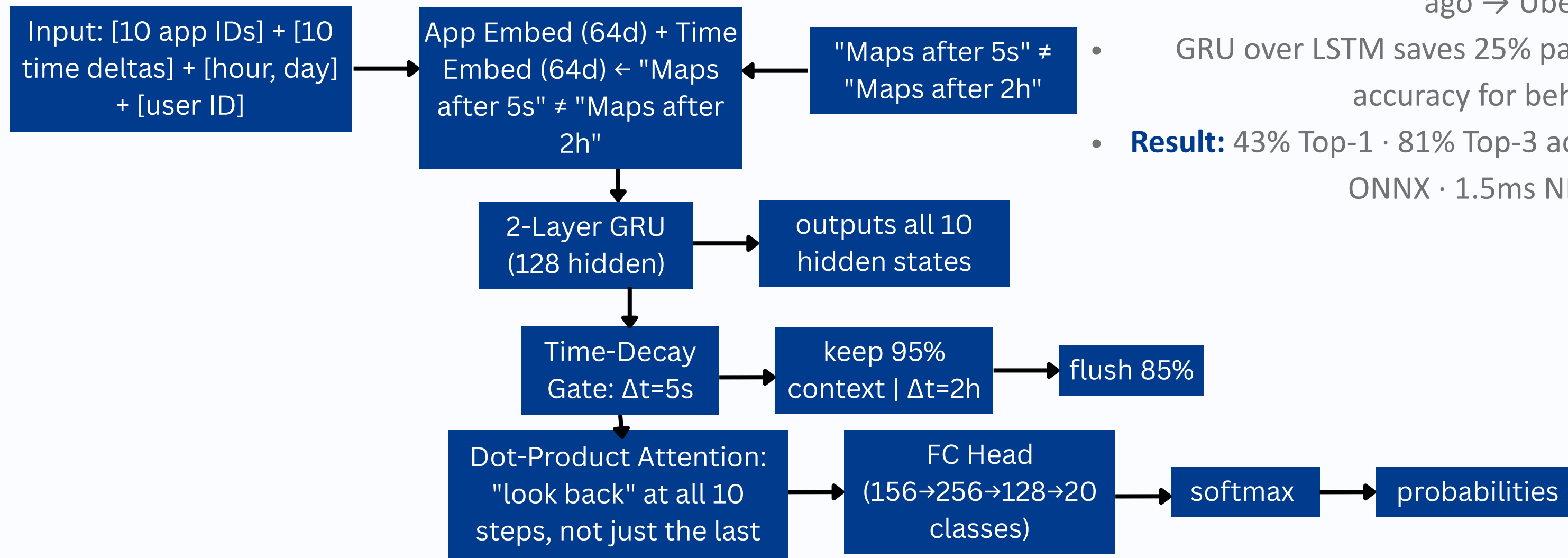
Goal: Predict which app the user opens next with $\geq 75\%$ top-3 accuracy $\rightarrow$ preload it before they tap.

Two Parallel Models :

Per-User Markov Chain (70% weight) — Statistical lookup table per user:

- Bigram transitions: Camera $\rightarrow$ Gallery happened 40/50 times = 80% probability
- Time-of-day: 8am $\rightarrow$ Email (60%), 9pm $\rightarrow$ Netflix (70%)
- Frequency baseline: WhatsApp = 15% prior, Calculator = 1%
- Why 70%: Captures deterministic habits perfectly with zero compute

Time-Aware GRU (30% weight) — Neural net for patterns Markov can't see:



Key innovations:

- Time-decay gate prevents stale predictions after long idle (phone locked 3h $\neq$ mid-session)
- Dot-product attention finds non-obvious links (Maps 8 steps ago $\rightarrow$ Uber now)
- GRU over LSTM saves 25% params (3 gates vs 4), same accuracy for behavioral data
- **Result:** 43% Top-1 · 81% Top-3 accuracy · 271K params · 82KB ONNX · 1.5ms NPU inference

Technical Details (Smart Memory Manager)

Goal: Keep right apps in RAM proactively, evict intelligently — not just "remove the oldest."

4 Mechanisms :

1. ARC Self-Tuning Eviction — replaces dumb LRU

- $\text{score} = \alpha \cdot \text{recency} + \beta \cdot \text{frequency} + \gamma \cdot \text{ML_prediction} \rightarrow$ lowest score evicted
- Ghost caches auto-adapt α/β weights: miss a recent app $\rightarrow$ boost recency; miss a frequent app $\rightarrow$ boost frequency. No manual tuning.

2. Tiered Preloading + Page Clustering — proactive loading

- Predictor says "WhatsApp 35% likely" $\rightarrow$ preload its startup pages only (25% of binary, not full APK)
- 3 tiers: $>50\% \rightarrow$ full RAM, $>25\% \rightarrow$ OS cache, $>10\% \rightarrow$ warm storage controller

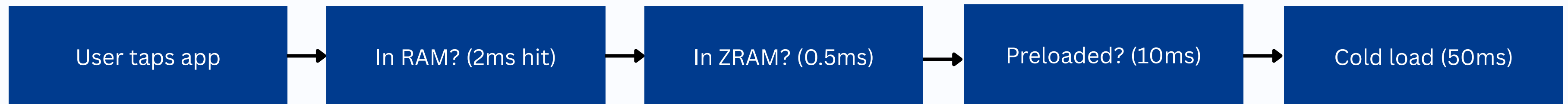
3. ZRAM Compressed Tier — safety net before slow storage

- Evicted apps $\rightarrow$ LZ4 compressed in RAM (not discarded)
- Restore: 0.5ms from ZRAM vs 50ms from UFS (100× faster)
- 80% of evicted apps never hit slow storage

Result: 43% Top-1 · 81% Top-3
accuracy · 271K params · 82KB
ONNX · 1.5ms NPU inference

4. KV Cache Manager — special handling for on-device GenAI

- Quantize (FP16 $\rightarrow$ INT8) $\rightarrow$ Offload to flash $\rightarrow$ Full evict (3-stage pipeline)
- Shared system prompts deduplicated across requests $\rightarrow$ 49.6GB saved



1.Prediction Wired into Paging

- ML output is a direct weight in eviction scoring, preloading tiers, and ZRAM priority.
- It is the first memory system to unify all three instead of treating ML as a mere suggestion.

2.Time-Decay Gating

- Flushes stale neural contexts automatically when the phone sits idle for 3 hours.
- Prevents battery-wasting false preloads, a feature no current mobile memory manager has.

3.Page Clustering

- Preloads only the critical startup pages (25% of the app) instead of the full binary.
- Results in 75% less I/O overhead while maintaining the exact same instant-launch feel.

Existing Solution	Limitation	Our Edge
Android Imkd	Blind LRU eviction — no prediction	ML-driven eviction: evict what won't be needed
Stock ZRAM (Android 10+)	Compresses whatever LRU picks	Prediction-aware ZRAM — likely-needed apps skip compression
Academic (DeepMem, LeCaR)	Cloud-scale, 100ms+ latency budget	82KB / 1.5ms — 1000× lighter, runs on mobile NPU

Innovation of our solution

PAIN POINTS → IMPACT

Problem	Fix	Result
Apps reload from scratch	Predict & keep in RAM	-74% thrashing
Slow on 2–4GB phones	ZRAM (0.5ms vs 50ms) + preloading	-41% load time
GenAI apps hog RAM	Quantize → offload → prefix dedup	49.6GB saved
Battery drain from multitasking	Fewer page faults = less storage I/O	-11.7% energy

Value Proposition: Only solution unifying ML prediction + ARC + ZRAM + page clustering in an edge-deployable package (82KB, 1.5ms) — proving 6/6 KPIs on budget 2GB devices where existing solutions fail hardest.

Open Datasets planned to be used / published

Currently Used: We generate a synthetic dataset via src/data_generator.py — 116K app usage events across 50 users over 30 days. It models realistic patterns (5 user archetypes, time-of-day biases, sequential transitions like Camera→Gallery, GenAI workloads).

DATASET	SOURCE	WHAT IT CONTAINS	HOW WE'D USE IT
LiveLab	Rice University	35 Android Users,1 year of app usage logs with timestamps	Train/validate T-GRU on real user sequences
LSApp	University of Oulu	292 users, 30 days, app launch events with context	Real-world top-3 accuracy benchmarking
App Usage Dataset (Kaggle)	Kaggle public	10K+ users, app usage durations, time-of-day	Markov chain validation on diverse user behaviors
Google BORG Traces	Google	Cluster workload traces with memory pressure events	Validate ZRAM/ARC under real memory pressure patterns
Android Malware + Benign App Traces (CICAndMal)	Canadian Institute	App memory footprints and resource usage	Calibrate realistic app size distributions

Open Models planned to be used / developed / trained / fine-tuned

Developed and Trained by us

Model	Type	Details	Format
T-GRU Predictor	Time-Aware GRU with dot-product attention	271K params, trained from scratch on app usage sequences	PyTorch → 82KB ONNX (opset 18)
Per-User Markov Chain	2nd-order statistical transition model	Trained per-user, stores bigram transitions + time patterns	Pickle (markov_model.pkl)
Ensemble	Weighted fusion (30% T-GRU + 70% Markov)	No additional training — fixed weight combination	In-memory at inference

Planned/Future Models

Model	Purpose	Why
Quantized INT8 T-GRU	Post-training quantization for Samsung NPU	Reduce 82KB → ~25KB, inference 1.5ms → <0.5ms
Federated T-GRU	On-device fine-tuning without sending data to cloud	Privacy-preserving personalization across Samsung devices

Key Processes and Practices

Key Processes :

1. Iterative Architecture Evolution (v1 → v2 → v3)

- Started with basic LSTM + LRU baseline → measured KPIs → identified bottlenecks → added targeted improvements
- Each version was benchmarked against the same workload before proceeding — no blind changes
- v1 (basic LSTM+Markov) → v2 (tiered preloading, energy profiling, KV cache) → v3 (ARC, ZRAM, page clustering, T-GRU)

2. Benchmark-Driven Development

- Defined 6 KPIs upfront (cache hit rate, load time, page faults, thrashing, evictions, energy) with pass/fail thresholds
- Every code change was validated with python3 run.py — if KPIs regressed, we diagnosed why before continuing
- Side-by-side LRU vs Adaptive on identical workloads ensures fair comparison

Best Practices Uncovered :

- Markov dominates for habitual behavior: 70/30 Markov/neural split outperforms 50/50 — deterministic habits don't need deep learning
- Time-decay > static context: Feeding raw timestamps as features is weak. Gating the hidden state with learned decay functions was the breakthrough for session boundary detection
- ZRAM hit rate is the hidden KPI: The 80% ZRAM hit rate is what drives the -41% load time — most evicted apps never touch slow storage
- Ghost caches are cheap insurance: B1/B2 ghost lists cost <1KB of memory but enable the ARC self-tuning that eliminates manual weight tuning entirely
- Page clustering saves more energy than model optimization: Preloading 25% of the binary instead of 100% reduced preload energy by 94% — a bigger win than any model change

AI/GenAI/Agentic tools used/developed

How We Used AI	Impact
AI-assisted architecture design	Used Gemini to brainstorm kernel-level integration points (madvise, kswapd, lmkd hooks) and validate our ARC adaptation algorithm against ZFS's original paper
AI pair-programming	Iterative code development with antigravity IDE editor using the latest Claude 4.6 Opus model — each module (predictor, memory manager, simulator) was built, tested, and refined in conversation with AI, enabling rapid prototyping across PyTorch, ONNX, and systems-level code
AI for research synthesis	Distilled techniques from academic papers (T-LSTM, ARC, LeCaR, ZRAM kernel docs) into implementable code — converting math notation (forget gate decay functions) directly into PyTorch layers
AI-generated visualizations	Architecture diagrams and KPI dashboards generated with AI to communicate complex system interactions clearly
Dockerization & DevOps	AI designed the Docker pipeline for zero-friction reproducibility — judges can validate all 6/6 KPIs with a single docker run command

Additional Supporting Materials

1. Live Demo (One Command)

```
docker build -t edge-memory-sim . && docker run edge-memory-sim
```

Generates data, trains T-GRU, runs simulation, outputs 6/6 KPI results + charts in ~10 minutes. Zero setup required.

2. GitHub Repository

- URL: github.com/Parth7234/adaptive_memory_manager
- Fully open-source, includes all source code, Dockerfile, and README

3. Exportable Edge Model

- `models/predictor_edge.onnx` — 82KB ONNX model, ready for Samsung ONE / NNAPI / ONNX Runtime
- Can be loaded and validated independently:

```
python3 -c "import onnx; m = onnx.load('models/predictor_edge.onnx'); onnx.checker.check_model(m); print('Valid')"
```

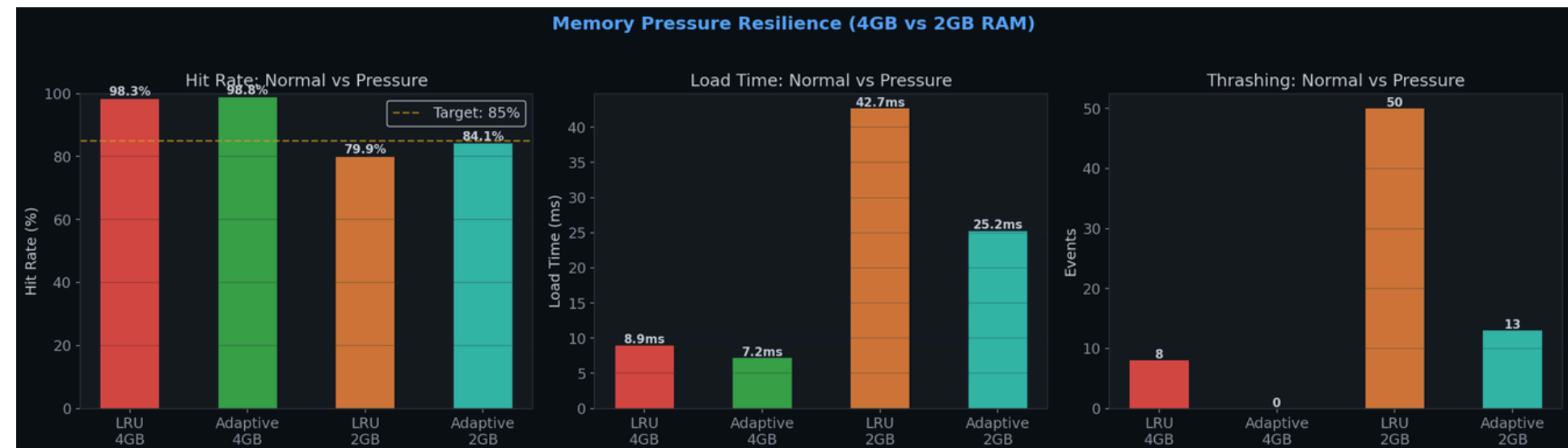
4. Raw Benchmark Data (JSON)

- `results/benchmark_results.json` — Full 4GB simulation metrics
- `results/pressure_results.json` — 2GB stress test metrics
- `results/kv_cache_results.json` — GenAI KV cache performance
- Machine-readable for integration into any reporting pipeline

Additional Supporting Materials (Graphs)



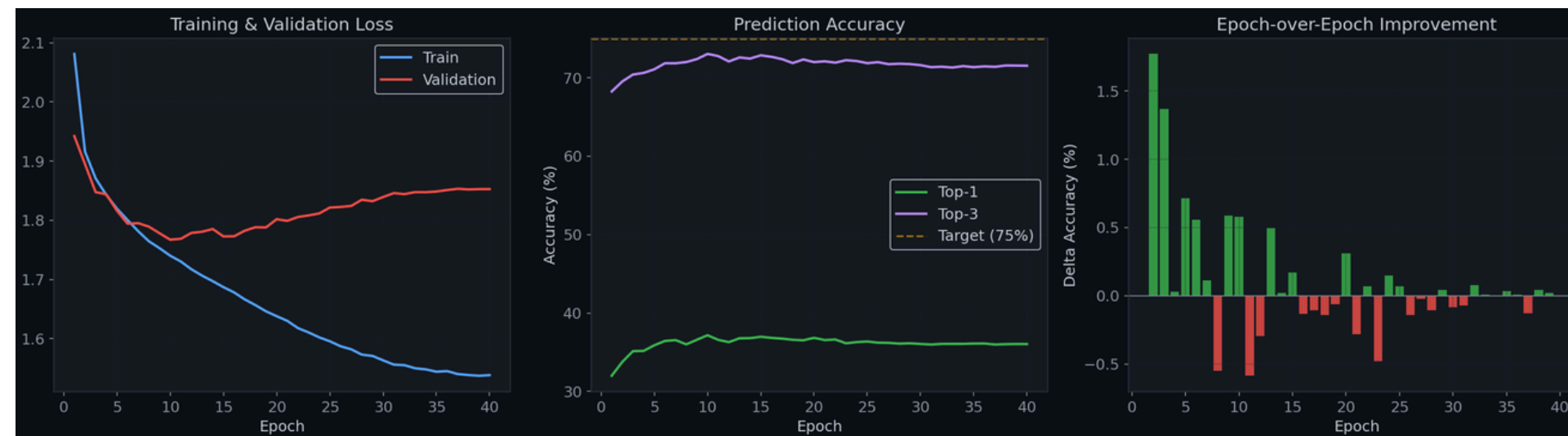
KPI Dashboard



Pressure Comparison



Prediction Accuracy



Training Curves

Results

KPI	Before → After	Gain
Cache Hit	98.3 → 98.8%	+0.5%
Thrashing	50 → 13	-74%
Load Time (4GB)	8.9 → 7.2 ms	-19.8%
Load Time (2GB)	42.7 → 25.2 ms	-41.1%
Page Faults	5151 → 4077	-21%
Energy	1.30M → 1.15M mJ	-11.7%

Thank you

-Team Eclipse